

Dockerfile Avanzado

Arturo Silvelo

Try New Roads

Introducción a Docker Compose avanzado

Docker Compose es una herramienta esencial para definir y gestionar aplicaciones multicontenedor en Docker. Permite describir la arquitectura de una aplicación, sus servicios, redes y volúmenes en un solo archivo YAML, facilitando la orquestación y el despliegue en diferentes entornos.

Aliases en YAML

En archivos YAML, como los usados por Docker Compose, los anchors (`&`) y alias (`*`) permiten reutilizar bloques de configuración, evitando duplicidad y facilitando el mantenimiento.

- **Anchor (`&`)**: Define un bloque reutilizable.
- **Alias (`*`)**: Inserta el bloque definido por el anchor.
- **Merge (`<<`)**: Permite combinar configuraciones.

Ventajas

- Menos repetición de código.
- Cambios centralizados y más fáciles de mantener.
- Configuraciones más limpias y legibles.

```
services:
  base-1:
    image: app-base
    container_name: base-1
    ports:
      - "3000:5000"
    environment: &env
      - PORT=5000
      - LOG_LEVEL=debug

  base-2:
    image: app-base
    container_name: base-2
    ports:
      - "4000:5000"
    environment: *env

  base-3:
    image: app-base
    container_name: base-3
    ports:
      - "5000:8000"
    environment: &env-list
      PORT: 8000
      LOG_LEVEL: warn

  base-4:
    image: app-base
    container_name: base-4
    ports:
      - "6000:8000"
    environment:
      <<: *env-list
      LOG_LEVEL: info
```

- Ejecución

```
docker compose -f 1.anchor/docker-compose.yaml up
```

- Verificación

```
docker exec base-1
docker exec base-2
docker exec base-3
docker exec base-4
```

- Resultado

```
LOG_LEVEL=debug
PORT=5000
```

- Limpieza

```
docker compose -f 1.anchor/docker-compose.yaml down
```

Healthcheck

El parámetro `healthcheck` en Docker Compose permite definir una comprobación periódica para saber si un servicio está funcionando correctamente. Esto ayuda a detectar fallos y a gestionar dependencias entre servicios.

El parámetro `healthcheck` puede definirse tanto en el `Dockerfile` como en cada servicio dentro de Docker Compose.

En ambos casos, se dispone de varias opciones de configuración:

- **test**: Comando que se ejecuta para comprobar la salud.
- **interval**: Frecuencia de la comprobación.
- **timeout**: Tiempo máximo de espera para la comprobación.
- **retries**: Número de intentos antes de marcar el servicio como unhealthy.
- **start_period**: Tiempo de gracia antes de empezar a comprobar.

depends_on

Es posible emplear `depends_on` junto con la condición `service_healthy` para garantizar que un servicio espere a que otro esté en estado saludable antes de iniciar su ejecución.

El servicio dependiente será creado y su contenedor iniciado, pero no comenzará su proceso principal hasta que el servicio del que depende alcance el estado saludable (`healthy`).


```
services:
  base-health:
    container_name: base-health
    image: app-base
    environment: &env-base
    USE_DB: true
    DB_HOST: postgres
    DB_PORT: 5432
    DB_USER: postgres
    DB_PASS: password
    DB_NAME: postgres
    ports:
      - "3000:3000"

  base-no-health:
    container_name: base-no-health
    image: app-base
    ports:
      - "4000:3000"
    environment:
      <<: *env-base
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 5s
      timeout: 10s
      retries: 3
    depends_on:
      postgres:
        condition: service_healthy

  base-health-conditional:
    container_name: base-health-conditional
    image: app-base
    depends_on:
      base-no-health:
        condition: service_healthy

  base-conditional:
    container_name: base-conditional
    image: app-base
    depends_on:
      - base-no-health

  postgres:
    container_name: postgres
    image: postgres:15
    environment: &db-env
    POSTGRES_PASSWORD: password
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 20s
      timeout: 10s
      retries: 3
```

Ejecución

```
docker compose -f 2.healthcheck/docker-compose.yaml up -d
```

Verificación

```
docker compose -f 2.healthcheck/docker-compose.yaml ps -a
```

Resultado

```
docker compose -f 02-compose/ejemplos/2.healthcheck/docker-compose.yaml ps -a
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
base-conditional	app-base	"docker-entrypoint.s..."	base-conditional	40 seconds ago	Up 19 seconds (healthy)	3000/tcp
base-health	app-base	"docker-entrypoint.s..."	base-health	40 seconds ago	Exited (1) 39 seconds ago	
base-health-conditional	app-base	"docker-entrypoint.s..."	base-health-conditional	40 seconds ago	Created	
base-no-health	app-base	"docker-entrypoint.s..."	base-no-health	40 seconds ago	Up 19 seconds (unhealthy)	0.0.0.0:4000->3000/tcp, [::]:4000->3000/tcp
postgres	postgres:15	"docker-entrypoint.s..."	postgres	40 seconds ago	Up 39 seconds (healthy)	5432/tcp

Limpieza

```
docker compose -f 2.healthcheck/docker-compose.yaml down
```

Gestión de entornos

En el desarrollo de aplicaciones, un entorno define el contexto en el que se ejecuta la aplicación: desarrollo, pruebas, integración o producción. Cada entorno puede requerir configuraciones, variables y servicios diferentes.

Docker y Docker Compose permiten gestionar estos entornos de varias formas:

- **Variables de entorno:** Permiten parametrizar el comportamiento de los servicios según el entorno. Se pueden definir directamente en el archivo Compose, en archivos `.env` o pasarlas desde el sistema.
- **Archivos Compose específicos:** Puedes crear archivos `docker-compose` adicionales (por ejemplo, `docker-compose.dev.yaml`, `docker-compose.prod.yaml`) para sobrescribir o extender la configuración base según el entorno.

- **Múltiples Dockerfile:** Es posible mantener diferentes Dockerfile (por ejemplo, Dockerfile.dev, Dockerfile.prod) adaptados a las necesidades de cada entorno, seleccionando el adecuado en la sección build del Compose.

Estas estrategias permiten adaptar fácilmente la infraestructura y el despliegue de la aplicación a cada fase del ciclo de vida.